

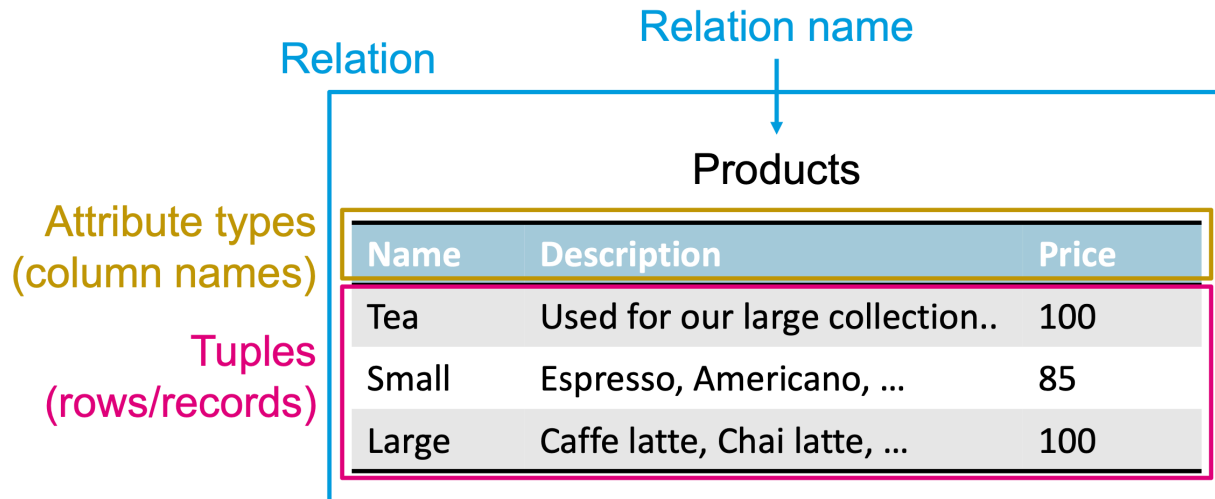
01 Relational Model

Overview

The Relational Model is one of the most widely used data models in database systems, offering a strong mathematical foundation based on set theory and first-order predicate logic. Proposed by Edgar F. Codd in 1970, it emphasizes simplicity, normalization, and logical consistency in data representation. It is the basis for many modern Relational Database Management Systems (RDBMS) such as Oracle, MySQL, and Microsoft SQL Server.

6.1.1 Basic Concepts

- **Relation Definition:** A database is a collection of relations, where each relation represents an entity type like products, suppliers, or employees.
- **Attributes:** Columns in a table represent attributes (e.g., product name, supplier ID).
- **Tuples:** Rows in a table correspond to individual entries or entities (e.g., a single product or supplier). For example, in a `Student` table, a tuple might represent one individual student with attributes such as ID, name, and phone number.
- **Domains:** Specify permissible values for an attribute (e.g., gender as male/female, or integers 1–9999 for product IDs).
- **Example:** A `SUPPLIER` relation includes attributes like `SUPNR`, `SUPNAME`, `SUPCITY`, and `SUPSTATUS`, forming tuples like `(69, Vinos del Mundo, San Francisco, 90)`.



- **Schema:** Relation name + list of attributes
 - Optional: attribute types
 - Products(Name: string, Description: string, Price: int)
- **Instance:** Tuples in a relation
- **Database:** Collection of relations (i.e. tables)

Products (Name: *string*, Description: *string*, Price: *int*)

Users (Name: *string*, Email: *string*, Password: *string*, Salt: *string*, CreatedOn: *date*)

Purchases (ProductName: *string*, userEmail: *string*, CreatedOn: *int*)

6.1.2 Formal Definitions

- **Relation R:** Formally defined as a subset of the Cartesian product of domains that correspond to its attributes.
- **Null Values:** Indicate missing, irrelevant, or inapplicable values in attributes.

- **Key Constraint:** Ensures all tuples in a relation are unique. No duplicate tuples are allowed.
-

6.1.3 Types of Keys

1. Superkeys and Keys:

- A **superkey** is any set of attributes that uniquely identifies a tuple in a relation. For instance, (Studentnr, Name, HomePhone) is a superkey, but redundant.
- A **key** is a minimal superkey with no redundant attributes, e.g., Studentnr in a STUDENT relation.

2. Candidate, Primary, and Alternative Keys:

- **Candidate Keys:** Multiple keys can uniquely identify a relation. For instance, a Product table may have both prodnr (product number) and prodname (product name) as candidate keys.
- **Primary Key:** The selected candidate key used for identification.
- **Alternative Keys:** Non-selected candidate keys.

3. Foreign Keys:

- Used to establish relationships between two tables by referring to the primary key of another relation, e.g., linking Order with Customer via CustomerID.
-

6.1.4 Relational Constraints

Relational constraints ensure data integrity:

- **Domain Constraints:** Attributes must contain values within their specified domain.
 - **Key Constraints:** Ensure no two tuples have the same value for the key attributes.
 - **Referential Integrity Constraints:** Enforce valid relationships between foreign keys and primary keys.
-

6.1.5 Example Relational Data Model

- Mapping real-world entities into relations:
 - Example: An employee may have multiple phone numbers, which can be modeled using two relations:
 - `EMPLOYEE(SSN, ename, address)`
 - `EMP_PHONE(PhoneNr, SSN)`
 - **Primary Key:** `SSN` uniquely identifies employees, while `(PhoneNr, SSN)` forms the primary key of `EMP_PHONE`.

```
cd /Users/yaboi/Desktop/Introduction\ to\ Database\
Systems/Normalization/Solver
```

```
javac io/bretty/solver/normalization/*.java
java io.bretty.solver.normalization.Main
```

Key Takeaways

- The relational model emphasizes simplicity, consistency, and adherence to mathematical principles, making it robust for managing structured data.
- Its key features, including domains, keys, and constraints, ensure data integrity and facilitate complex relationships between entities.
- The model's normalization process avoids redundancies and anomalies in data representation, setting the groundwork for advanced database operations.